



# **The Class P (7.2)**

---

COT 4210 Discrete Structures II  
Summer 2025  
Department of Computer Science  
Dr. Steinberg

# The Path Problem: Brute Force

Given a directed graph  $G$  and nodes  $s$  and  $t$ :

$$PATH = \{\langle G, s, t \rangle \mid \text{A path exists from } s \text{ to } t \text{ in } G\}$$

Observe by counting that a path can never be longer than the number of nodes in  $G$ .

**M = On input  $\langle G, s, t \rangle$ :**

1. Calculate  $n$ , the number of nodes in  $G$ .
2. Iterate through each unique sequence  $x$  of nodes of length less than or equal to  $n$ .
  - a. If  $x$  is a directed path from  $s$  to  $t$ , accept.
3. If we have not accepted, reject.

# The Path Problem: Brute Force

Given a directed graph  $G$  and nodes  $s$  and  $t$ :

$$PATH = \{\langle G, s, t \rangle \mid \text{A path exists from } s \text{ to } t \text{ in } G\}$$

Observe by counting that a path can never be longer than the number of nodes in  $G$ .

$M =$  On input  $\langle G, s, t \rangle$ :

1. Calculate  $n$ , the number of nodes in  $G$ .
2. Iterate through each unique sequence  $x$  of nodes of length less than or equal to  $n$ .
  - a. If  $x$  is a directed path from  $s$  to  $t$ , accept.
3. If we have not accepted, reject.

This *does* work to find a path, so it's enough to prove decidability. But it's *really slow*.

- Step 2 is the interesting step, and it has to run for every sequence. How many of those are there?

$$\sum_{i=0}^n (\text{nPr}(n, i)) \cong n^n$$

This is an example of a **brute force search** – and of why we try not to do them.

- Brute force searches tend to be exponential algorithms.
- Exponential algorithms grow too fast to be useful...
- ...at least for anything other than the smallest data sets.

# The Path Problem: A Better Solution

Given a directed graph  $G$  and nodes  $s$  and  $t$ :

$$PATH = \{\langle G, s, t \rangle \mid \text{A path exists from } s \text{ to } t \text{ in } G\}$$

**M = On input  $\langle G, s, t \rangle$ :**

1. Mark node  $s$ .
2. Repeat until no new nodes are marked:
  - a. Scan over each edge  $(a, b)$  of  $G$ :
    - i. If node  $a$  is marked and node  $b$  is unmarked, mark node  $b$ .
3. If node  $t$  is marked, accept, else reject.

# The Path Problem: A Better Solution

Given a directed graph  $G$  and nodes  $s$  and  $t$ :

$$PATH = \{\langle G, s, t \rangle \mid \text{A path exists from } s \text{ to } t \text{ in } G\}$$

M = On input  $\langle G, s, t \rangle$ :

1. Mark node  $s$ .
2. Repeat until no new nodes are marked:
  - a. Scan over each edge  $(a, b)$  of  $G$ :
    - i. If node  $a$  is marked and node  $b$  is unmarked, mark node  $b$ .
3. If node  $t$  is marked, accept, else reject.

Breadth-first search gets us where we need to go in much better time.

Step 2 is again the only interesting step.

- Observe that there are  $n$  nodes.
- Observe that there are at most  $n^2$  edges.

So it runs in about  $n^3$  time in the number of nodes – not great\*, but a lot better than exponential, and enough for us to declare that there is a polynomial-time algorithm for PATH.

(\* Yes, we can do better. Look up **Dijkstra's Algorithm** if you want to know how.)

# Polynomial Equivalence

- We've said before that reasonable computation models are equivalent in what they can decide.
- When it comes to performance, we can make a similar statement. It turns out that:

All reasonable **deterministic** computational models are **polynomially equivalent**.

- By polynomially equivalent, we mean that all of them can simulate each other with only a polynomial increase in running time.
- We saw this when we simulated a multi-tape Turing machine with a single-tape TM.
- We also saw that **nondeterministic** models can introduce much more profound differences.
- From this point forward, we are concerned primarily with the aspects of time complexity unaffected by polynomial equivalence.
  - In other words, we care about the difference between  $n^2$  and  $2^n$ , but not about the difference between  $n^2$  and  $n^3$ .
  - This means that all our computational models become equivalent again, and allows us to define an immensely important class of languages.

# The Class P

- P is the class of languages that are **decidable in polynomial time by a deterministic, single-tape Turing machine**. In other words:

$$P = \bigcup_k [\text{TIME}(n^k)] = [\text{TIME}(n) \cup \text{TIME}(n^2) \cup \text{TIME}(n^3) \cup \text{TIME}(n^4) \cup \dots]$$

- Several observations follow immediately. The first set of them is about how durable P is:
  - Remember, all reasonable deterministic computational models are polynomially equivalent.
  - That means that, for practical purposes, **P is invariant across deterministic computational models**. To express this concisely, we say that P is a **robust** class.
- The second set is about how useful P is:
  - For practical purposes, and ignoring intentionally pathological cases, **P corresponds to the class of problems we can really solve, with real computers, in the real world, using a realistic amount of time and effort**.
  - This isn't perfect –  $n^{64}$  problems aren't very workable, and large enough  $n$  still gives us problems.
  - But once you've got a problem into P, you can start optimizing the algorithm and implementation.
  - We're *good* at that.

# Greatest Common Divisor

---

Given two natural numbers  $x$  and  $y$ , their greatest common divisor is the largest natural number that evenly divides both of them.

Let's analyze the Euclidean GCD algorithm:

$\text{GCD}(x, y) =$  **On input  $\langle x, y \rangle$ :**

1. Repeat until  $y = 0$ :
  - a. Let  $x = x \bmod y$ .
  - b. Exchange  $x$  and  $y$ .
2. Output  $x$ .



# Greatest Common Divisor

Given two natural numbers  $x$  and  $y$ , their greatest common divisor is the largest natural number that evenly divides both of them.

Let's analyze the Euclidean GCD algorithm:

$\text{GCD}(x, y) =$  **On input  $\langle x, y \rangle$ :**

1. Repeat until  $y = 0$ :
  - a. Let  $x = x \bmod y$ .
  - b. Exchange  $x$  and  $y$ .
2. Output  $x$ .

This is just the Euclidean Algorithm, so we'll leave its correctness to a million different math books and stick to analyzing the complexity.

Observe that:

- Every execution of  $x = x \bmod y$  must cut  $x$  at least in half.
- We exchange  $x$  and  $y$  every step.

So the maximum number of repetitions of step 1 is  $(2 \log_2 n)$ .

- That's proportional to the length of a binary representation.
- We can call this  $O(n)$  in the *length* (not value!) of  $x$  and  $y$ .

## Relative Prime Numbers

Given two integers  $x$  and  $y$ , they are relatively prime if 1 is the largest integer that evenly divides both of them.

$RELPRIME = \{ \langle x, y \rangle \mid x \text{ and } y \text{ are relatively prime} \}$

We can use the GCD algorithm to decide this language:

**On input  $\langle x, y \rangle$ :**

1. Let  $z = \text{GCD}(x, y)$ .
2. If  $z = 1$ , accept. Otherwise, reject.

Since we already know *GCD* executes in polynomial time,  $RELPRIME \in P$  follows immediately.



# Acknowledgement

---

Some Notes and content come from Dr. Gerber, Dr. Hughes, and Mr. Guha's COT4210 class and the Sipser Textbook, *Introduction to the Theory of Computation*, 3<sup>rd</sup> ed., 2013